

ScoreWizz Platform

Cricket Tournament Engine, Live Scoring PWA & Cloud Deployment Report

Author: **Suraj Gavada** Application: **ScoreWizz v2.0** Repository: **surajgavada/scorewizz** Host Target: **Render (Python 3)**

1. Executive Summary

ScoreWizz is an offline-ready, zero-dependency Progressive Web Application (PWA) designed for multi-format cricket tournament management, live ball-by-ball scorekeeping, automated Net Run Rate (NRR) standings calculation, player statistics aggregation, and instant cloud deployment.

Engineered without bulky frontend frameworks or third-party web server packages, the platform achieves near-zero latency, runs completely offline, and deploys effortlessly across cloud environments like **Render**.

2. What Is Used? (Technology Stack Breakdown)

Layer / Tier	Technology	Role & Functionality
Backend Server	Python 3 (<code>http.server</code>)	Lightweight multi-threaded REST HTTP API server, routing static assets, handling JSON payloads, and dynamic <code>\$PORT</code> cloud binding.
Database Engine	SQLite 3 (<code>scorewizz.db</code>)	ACID-compliant relational database storing tournaments, teams, squads, fixtures, ball-by-ball logs, and historical scorecards.
Frontend Core	Vanilla JavaScript (ES6+)	Modular client-side state machine (~6,500 lines) managing live scoring, Playing 11 rosters, tie-breakers, awards calculation, and UI updates.
User Interface	HTML5 & Modern CSS3	Semantic SPA structure with 7 distinct viewports, CSS design tokens, high-contrast light/dark themes, and zero Unicode emojis.
Offline & PWA	Service Worker & Cache API	Caches static assets (<code>sw.js</code> v66) and syncs real-time state with <code>localStorage</code> for 100% offline match continuity.
Cloud Deployment	Render Cloud Platform	Automated continuous deployment from GitHub using <code>render.yaml</code> , <code>Procfile</code> , and <code>requirements.txt</code> with free SSL/HTTPS.

3. Why Was It Used? (Engineering Decisions & Rationales)

Zero Third-Party Dependencies

Efficiency

Utilizing Python Standard Library (`http.server` , `sqlite3` , `json` , `urllib`) eliminates security vulnerabilities, package rot, and heavy `npm` / `pip` overhead, ensuring instant boot times (<1s) and effortless builds on free-tier cloud instances.

Vanilla JS vs. Heavy Frameworks

Performance

Avoiding React/Vue/Angular removes virtual DOM memory footprints and build steps. The app loads in under 100ms and executes natively on low-end mobile devices used on outdoor cricket grounds.

Dual-Storage Architecture

Resilience

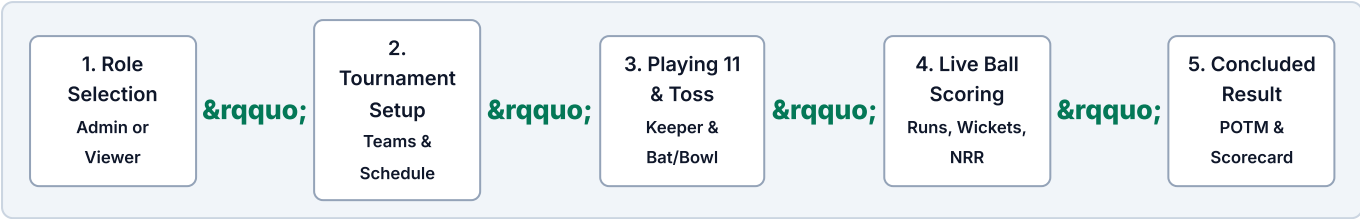
Real-time snapshots are maintained synchronously in browser `localStorage` while asynchronously syncing to SQLite via REST APIs. If ground internet drops mid-match, zero scoring data is lost.

High-Contrast & Emoji-Free UI

Usability

Strict light-mode contrast (Emerald `#047857` , Coral `#e11d48` , Slate `#1e293b`) guarantees full sunlight readability. Replacing Unicode emojis with vector SVGs ensures consistent rendering across all devices.

4. How It Works (Core Subsystems & Data Flow)



A. Role-Based Access & Workspace Isolation

- Admin Authentication:** Admin enters a name (e.g. `Suraj`). Tournaments are strictly filtered by creator/owner (`owner === adminName`), ensuring multi-user isolation so new admins start with a clean workspace.
- Viewer Mode (Read-Only):** Viewers receive read-only access. All scoring pads, toss controls, reset buttons, and delete actions are hidden (`.role-viewer .admin-only { display: none !important; }`).

B. Tournament Scheduling & Knockout Advancement

- Round-Robin Generator:** Generates balanced home/away schedules using round-robin circle algorithms.
- Knockout Tree Engine:** Dynamically models elimination brackets (Quarter-Finals, Semi-Finals, Grand Final). Concluding a match automatically slots the winning team into the next bracket fixture.

C. Playing 11 Selection & Detailed Dismissals

- Squad Validation:** Enforces exactly 11 active players, designating Captain, Vice-Captain, and Wicketkeeper (who is also permitted to bowl).
- Dismissal Engine:** Captures exact dismissal parameters: *Caught* (selects fielder from opposing 11), *Run Out* (records fielder and run-out batter), *Bowled*, *LBW*, *Stumped*, *Hit Wicket*, *Retired Hurt*.

D. Real-Time Net Run Rate (NRR) & Leaderboards

Standings and Net Run Rate are recalculated dynamically upon every match conclusion using the standard ICC formula:

`Net Run Rate (NRR) = (Total Runs Scored / Overs Faced) - (Total Runs Conceded / Overs Bowled)`

Leaderboards automatically rank tournament MVPs, **Orange Cap** (Top Run Scorers), and **White & Purple Cap** (Top Wicket Takers).

E. Post-Match Concluded Mode & Bowler POTM Awards

- Direct Result Mode (No Rematch):** When a match concludes, all interactive scoring controls are hidden and replaced with a direct **View Result** action. Concluded matches directly open the official scorecard without prompt popups.
- Comprehensive Bowler POTM Statistics:** When Player of the Match is awarded to a bowler, figures display Wickets/Runs (e.g. `3/24`), Overs & Maidens (`4.0 ov, 1 maiden`), and Economy Rate (`6.00`).

5. Cloud Deployment & Hosting Process (Render)

File / Config	Value / Content	Deployment Purpose
render.yaml	type: web, runtime: python	Declarative infrastructure blueprint for automatic Render service setup.
requirements.txt	Standard Library declaration	Ensures cloud Python package installer verifies zero dependencies.
Procfile	web: python app.py	Directs the host runtime process to launch the Python HTTP server.
runtime.txt	python-3.11.9	Locks the deployment environment to Python 3.11 LTS
PORT Binding	os.environ.get('PORT', 3000)	Dynamically binds to Render's allocated external port with automatic SSL/HTTPS.